

Práctica 1. Entorno de trabajo y técnicas básicas

Ejemplos de uso de librerías para ciencia de datos

Curso 2026–2027

NumPy proporciona arrays multidimensionales y operaciones matemáticas sobre ellos. Estos arrays representan vectores, matrices y datos de más dimensiones. Las operaciones vectorizadas permiten trabajar sin recorrer elemento a elemento desde Python.

<https://numpy.org/>

SciPy complementa a NumPy con algoritmos para álgebra lineal, matrices dispersas, procesamiento de señales e imágenes, optimización, transformadas de Fourier y estadística.

En la Parte 1 se utilizan distribuciones de probabilidad, contrastes, estimación de densidad y distancias.

<https://scipy.org/>

Pandas proporciona las estructuras `Series` y `DataFrame` para manejar datos tabulares con columnas de distintos tipos.

Incluye operaciones para leer datos, extraer subconjuntos, tratar valores perdidos, combinar tablas y resumir información.

<https://pandas.pydata.org/>

Scikit-learn reúne herramientas de aprendizaje automático con una interfaz común. Contiene módulos de preprocesamiento, aprendizaje supervisado y no supervisado, selección de modelos, validación y métricas.

<https://scikit-learn.org/stable/>

Matplotlib crea gráficos a partir de arrays y tablas. La interfaz pyplot proporciona funciones de dibujo similares a las de MATLAB.

<https://matplotlib.org/>

- Distribuciones de probabilidad: `scipy-1.py` y `scipy-2.py`.
- Contrastes y densidades: `scipy-3.py` y `scipy-4.py`.
- Distancias entre instancias: `scipy-5.py`.

Antes de empezar, abra la carpeta de la práctica en Visual Studio Code, seleccione el intérprete y ejecute `comprobar_entorno.py`.

scipy-1: adición de ruido a una imagen

```
from scipy.stats import norm

imagen = ascent().astype(float)

# Si no se consigue hacer el pip de pooch en el PC de clase
# import pickle
# from pathlib import Path
#
# CARPETA = Path(__file__).resolve().parent
# with (CARPETA / "ascent.dat").open("rb") as archivo:
#     imagen = np.array(pickle.load(archivo), dtype=float)

ruido = norm.rvs(loc=0, scale=16, size=imagen.shape,
                 random_state=42)
imagen_ruido = np.clip(imagen + ruido, 0, 255)

# %% Celda 1
fig, ejes = plt.subplots(1, 2)
ejes[0].imshow(imagen, cmap="gray", vmin=0, vmax=255)
ejes[0].set_title("Original")
ejes[1].imshow(imagen_ruido, cmap="gray", vmin=0, vmax=255)
ejes[1].set_title("Ruido: desviación 16")
for eje in ejes:
    eje.axis("off")
plt.tight_layout()
```

Original



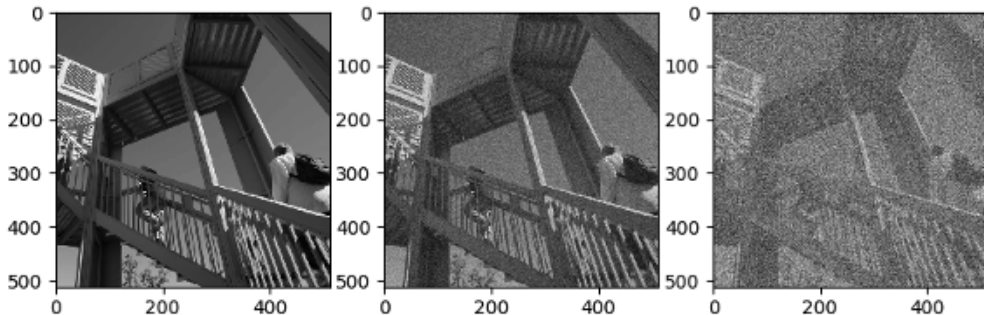
Ruido: desviación 16



Ejercicio 1

Añada una tercera imagen con ruido gaussiano de desviación típica 64.

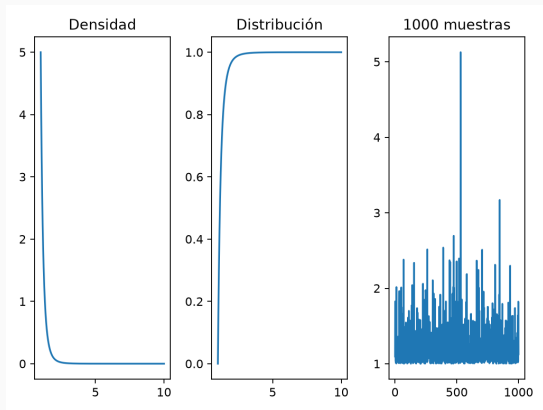
Muestre juntas la imagen original y las dos versiones ruidosas. Use en las tres la misma escala de grises, con $v_{\min}=0$ y $v_{\max}=255$, para que la comparación sea válida.



scipy-2: distribuciones de probabilidad

```
import matplotlib.pyplot as plt
import numpy as np
from scipy.stats import pareto

x = np.linspace(1, 10, 1000)
fig, ejes = plt.subplots(1, 3)
muestras = pareto.rvs(5, size=1000, random_state=42)
ejes[0].plot(x, pareto.pdf(x, 5))
ejes[0].set_title("Densidad")
ejes[1].plot(x, pareto.cdf(x, 5))
ejes[1].set_title("Distribución")
ejes[2].plot(muestras)
ejes[2].set_title("1000 muestras")
plt.tight_layout()
plt.show()
```

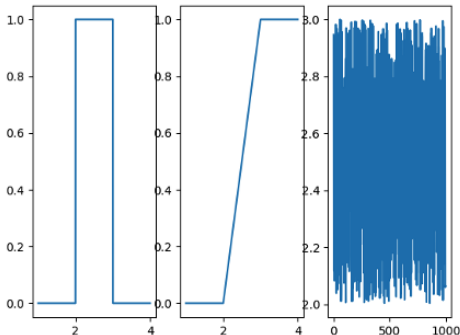


La figura compara la densidad, la distribución acumulada y una muestra de la misma distribución de Pareto.

Ejercicio 2

Dibuje la función de densidad, la función de distribución y 1000 muestras de una distribución uniforme en $[2, 3]$.

En `scipy.stats.uniform`, `loc` indica el inicio del intervalo y `scale` su anchura. En este caso se utilizan `loc=2` y `scale=1`.



scipy-3: datos emparejados

25 sujetos probaron un actuador en ambos sentidos. Las dos observaciones de una misma posición pertenecen a la misma persona.

El contraste se aplica a las diferencias:

$$d_i = \text{izquierda}_i - \text{derecha}_i.$$

La hipótesis nula afirma que la media de esas diferencias es cero.

Subject	1	2	3	4	5	6	7	8	9	10
Right thread	113	105	130	101	138	118	87	116	75	96
Left thread	137	105	133	108	115	170	103	145	78	107
Subject	11	12	13	14	15	16	17	18	19	20
Right thread	122	103	116	107	118	103	111	104	111	89
Left thread	84	148	147	87	166	146	123	135	112	93
Subject	21	22	23	24	25					
Right thread	78	100	89	85	88					
Left thread	76	116	78	101	123					

scipy-3: contraste t sobre las diferencias

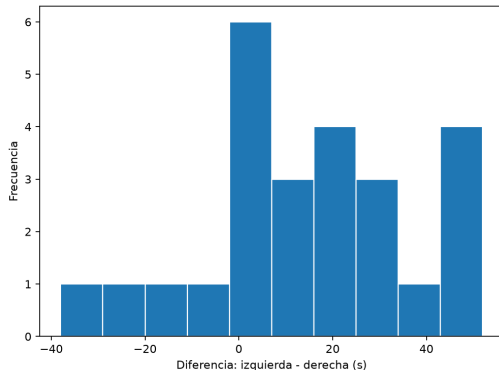
```
izquierda = np.array([
    137, 105, 133, 108, 115, 170, 103, 145, 78, 107,
    84, 148, 147, 87, 166, 146, 123, 135, 112, 93,
    76, 116, 78, 101, 123,
])

diferencias = izquierda - derecha
print("Media:", diferencias.mean())
print("Desviación típica muestral:", diferencias.std(ddof=1))

plt.hist(diferencias, edgecolor="white")
plt.xlabel("Diferencia: izquierda - derecha (s)")
plt.ylabel("Frecuencia")
plt.show()

resultado = ttest_1samp(diferencias, popmean=0)
print(f"t = {resultado.statistic:.3f}; p = {resultado.pvalue:.4f}")

if resultado.pvalue < 0.05:
    print("Rechazamos H0: hay evidencia de una diferencia media.")
else:
    print("No rechazamos H0: la evidencia no es suficiente.")
```



Ejercicio 3

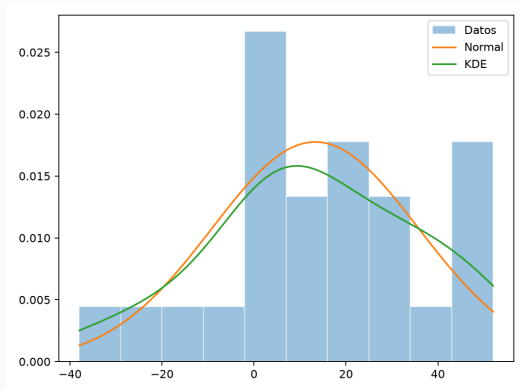
Repita el análisis con el contraste de rangos con signo de Wilcoxon aplicado a las diferencias emparejadas.

Indique el p-valor y compare la conclusión con la del contraste t, teniendo en cuenta que los supuestos de ambos contrastes son distintos.

scipy-4: ajuste y estimación de densidad

```
diferencias = izquierda - derecha
media, desv = ss.norm.fit(diferencias)
x = np.linspace(diferencias.min(), diferencias.max(), 1000)
normal = ss.norm.pdf(x, media, desv)
kde = ss.gaussian_kde(diferencias)

plt.hist(
    diferencias,
    density=True,
    alpha=0.45,
    edgecolor="white",
    label="Datos",
)
plt.plot(x, normal, label="Normal")
plt.plot(x, kde(x), label="KDE")
plt.legend()
plt.tight_layout()
plt.show()
```



La normal impone una familia paramétrica. La estimación kernel obtiene una curva suave sin imponer esa forma.

Ajuste también una distribución de Cauchy a las diferencias.

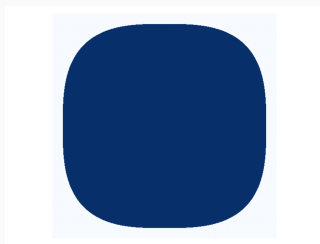
Añada su densidad al gráfico e identifique mediante una leyenda el histograma, la normal, la Cauchy y la estimación kernel.

scipy-5: distancia de Minkowski

```
rejilla = np.meshgrid(
    np.linspace(-1.1, 1.1, 512),
    np.linspace(-1.1, 1.1, 512),
    indexing="ij",
)
X, Y = rejilla

def en_bola(x, y, p):
    return minkowski([x, y], [0, 0], p) <= 1
def bola(p):
    return np.vectorize(en_bola)(X, Y, p)

plt.imshow(
    bola(3),
    extent=[-1.1, 1.1, -1.1, 1.1],
    origin="lower",
    cmap="Blues",
)
plt.gca().set_aspect("equal")
plt.axis("off")
plt.show()
```



$$d_p(x, y) = \left(\sum_j |x_j - y_j|^p \right)^{1/p}$$

Para $p = 1$ se obtiene Manhattan y para $p = 2$ la euclídea. El límite cuando $p \rightarrow \infty$ es Chebyshev.

Ejercicio 5

Dibuje la bola de radio 1 para las distancias de Minkowski de órdenes 1, 2 y 4.

Dibuje también las bolas de radio 1 para las distancias euclídea, Chebyshev y Manhattan. Relacione cada distancia con el orden correspondiente de Minkowski.

Flujo de trabajo utilizado en la práctica:

1. cargar el conjunto de datos;
2. preprocesar o seleccionar variables;
3. aprender el modelo mediante `fit()`;
4. obtener predicciones mediante `predict()`;
5. evaluarlas con una métrica y datos no usados durante el ajuste.

Boston: datos y limitaciones éticas

El conjunto estudia el valor de las viviendas mediante 13 variables: CRIM, ZN, INDUS, CHAS, NOX, RM, AGE, DIS, RAD, TAX, PTRATIO, B y LSTAT. El objetivo es MEDV.

Se entrega `boston.csv` porque scikit-learn retiró `load_boston`. La variable B se construyó a partir de una hipótesis racial cuya validez no se demostró adecuadamente.

El conjunto se conserva para estudiar el código histórico y discutir cómo el origen de los datos condiciona su uso. No debe emplearse para extraer conclusiones sociales o causales.

Fuente: https://scikit-learn.org/1.0/modules/generated/sklearn.datasets.load_boston.html

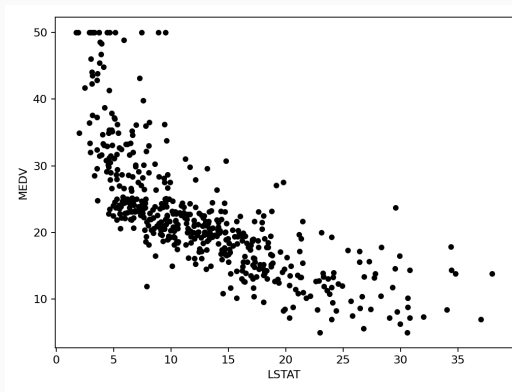
Boston: datos y limitaciones éticas

1. CRIM: per capita crime rate by town
2. ZN: proportion of residential land zoned for lots over 25,000 sq.ft.
3. INDUS: proportion of non-retail business acres per town
4. CHAS: Charles River dummy variable (= 1 if tract bounds river; 0 otherwise)
5. NOX: nitric oxides concentration (parts per 10 million)
6. RM: average number of rooms per dwelling
7. AGE: proportion of owner-occupied units built prior to 1940
8. DIS: weighted distances to five Boston employment centres
9. RAD: index of accessibility to radial highways
10. TAX: full-value property-tax rate per \$10,000
11. PTRATIO: pupil-teacher ratio by town
12. B: $1000(B_k - 0.63)^2$ where B_k is the proportion of blacks by town
13. LSTAT: % lower status of the population
14. MEDV: Median value of owner-occupied homes in \$1000's

Representación de un modelo con una única entrada

```
selector = SelectKBest(score_func=f_regression, k=1)
X = selector.fit_transform(X_full, y)
variable = X_full.columns[selector.get_support()][0]
print("Variable seleccionada:", variable)

plt.figure()
plt.scatter(X[:, 0], y, color="black", s=16)
plt.xlabel(variable)
plt.ylabel("MEDV")
guardar("sklearn-1-univariante-datos.png")
```

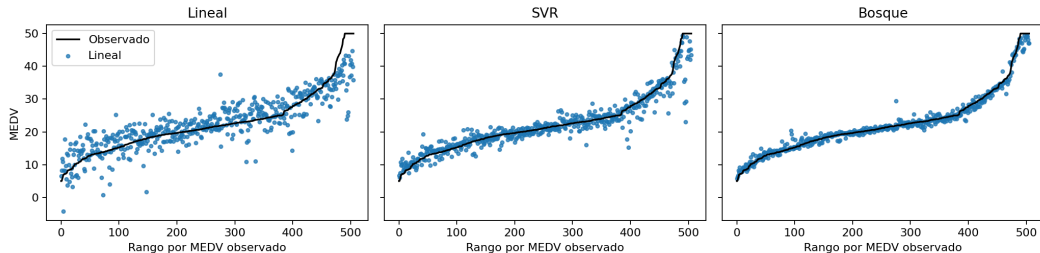


SelectKBest(k=1) selecciona LSTAT. Si seleccionamos solamente esa variable de entrada, puede hacerse un scatter con esa variable en el eje horizontal.

Representación de un modelo con varias entradas

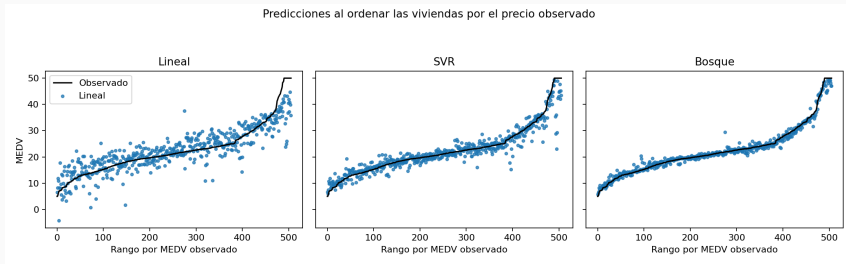
Con 13 variables no existe un único eje X para un scatter bidimensional.
`sklearn-1-multivariate.py` sustituye ese eje por el rango de las viviendas.

Predicciones al ordenar las viviendas por el precio observado



Al ordenar por MEDV, la curva negra siempre crece. La separación entre observación y predicción permite observar el comportamiento en viviendas baratas y caras.

Representación de un modelo con varias entradas



El buen ajuste es engañoso porque los datos se están dibujando sobre el mismo conjunto en el que se han ajustado.

Un mal ajuste significa que el modelo no es lo bastante plástico, pero aún no estamos midiendo el sobreentrenamiento.

Ejercicio 6: representación de un modelo con varias entradas

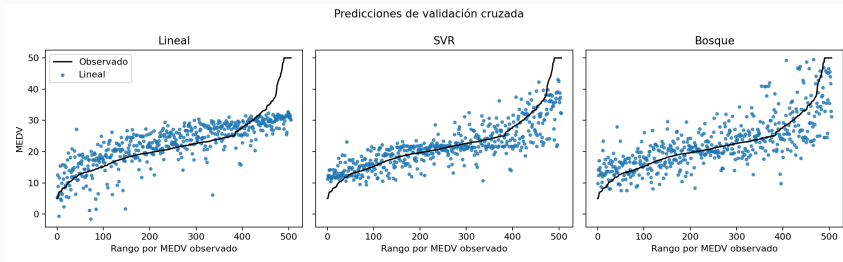
Modifica `sklearn-1-multivariate.py`:

1. Añade un `SelectKBest` a los modelos de varias variables y compara los resultados de estos para valores de `k` diferentes de 1 y de 13
2. Prueba con diferentes tipos de kernel y valores de parámetro `C` en `SVR()`

En el siguiente ejercicio veremos cómo es el ajuste de estos modelos en un conjunto de datos no usados durante el entrenamiento.

sklearn-2: validación cruzada

```
RandomForestRegressor(...),  
)  
}  
  
# %% Validación cruzada con las mismas particiones  
particiones = KFold(n_splits=10, shuffle=True, random_state=42)
```



Si usamos validación cruzada y mostramos las predicciones de cada modelo en vez de los datos de entrenamiento, la gráfica cambia y las conclusiones son otras.

Ejercicio 7

Completa el código de `sklearn-2.py` y contesta las siguientes preguntas:

- ¿Cuál de los modelos obtiene menor MSE mediante validación cruzada: LIN, SVR o Random Forest?
- ¿Cómo cambia el resultado si se utilizan todas las variables en lugar de la seleccionada con $k = 1$?
- Examine las predicciones ordenadas por MEDV. ¿Se comportan igual los modelos en las viviendas baratas, en las de precio intermedio y en las más caras?

- Entrada de datos.
- Visión general del conjunto de datos.
- Manejo básico de valores perdidos.
- Combinación de DataFrames.
- Selección de datos en un DataFrame.

Los cinco archivos conservan esta selección y este orden.

Pandas: entrada de datos

```
# %% Lectura del conjunto completo desde CSV
df = pd.read_csv(CARPETA / "Churn_Modelling_sintetico.csv")
print(df.head())

# %% Lectura selectiva de columnas y filas
columnas = ["CustomerId", "CreditScore", "NumOfProducts"]
seleccion = pd.read_csv(
    CARPETA / "Churn_Modelling_sintetico.csv", usecols=columnas, nrows=500
)
print(seleccion.head())
print(seleccion.shape)

# %% Lectura y escritura de Excel
df_excel = pd.read_excel(CARPETA / "Churn_Modelling_sintetico.xlsx")
df_excel.head(100).to_excel(CARPETA / "miarchivo.xlsx", index=False)
print(df_excel.head())
```

pandas-1.py muestra lectura completa y selectiva de CSV, lectura y escritura de Excel y creación de un DataFrame desde un diccionario.

Pandas: visión general

```
# %% Descripción estadística de los datos numéricos
# Los valores NaN no se incluyen en count ni en los estadísticos.
print(df.describe())

# %% Acceso a una columna y recuento de valores
print(df["c"])
print(df["c"].value_counts(dropna=False))

# %% Tipos de datos inferidos
print(df.dtypes)
```

`describe()` resume las columnas numéricas. `value_counts()` cuenta los valores de una columna y `dtypes` permite revisar los tipos inferidos.

Pandas: valores perdidos

```
# %% Detección y recuento
print(df.isna())
print(df.notna())
print(df.isna().any())
print(df.isna().sum())

# %% Normalización de símbolos que representan ausencia
normalizado = df.replace(["?", "--"], np.nan)
print(normalizado)

# %% Eliminación: cada operación parte del mismo DataFrame
print(normalizado.dropna(axis=0, how="all"))
print(normalizado.dropna(axis=1, how="any"))
print(normalizado.dropna(axis=0, how="any"))

# %% Imputación mediante una constante o un estadístico
print(normalizado.fillna(25))
con_media = normalizado.copy()
con_media["column_a"] = con_media["column_a"].fillna(con_media["column_a"].mean())
print(con_media)

# %% Propagación del valor anterior o siguiente
# Solo tiene sentido cuando el orden de las filas es relevante.
print(normalizado.ffill())
print(normalizado.bfill())
print(normalizado.ffill(limit=1))
```

Pandas: combinación de DataFrames

```
# %% Concatenación por filas
print(pd.concat([df1, df2]))
print(pd.concat([df1, df2], ignore_index=True))

# %% Concatenación por columnas usando los índices actuales
print(pd.concat([df1, df2], axis=1))

# %% Efecto de índices no coincidentes
df2_desplazado = df2.copy()
df2_desplazado.index = [3, 4, 5, 6]
print(pd.concat([df1, df2_desplazado], axis=1, join="inner"))
print(pd.concat([df1, df2_desplazado], axis=1, join="outer"))
```

La concatenación por filas conserva los índices originales salvo que se indique `ignore_index=True`. Al concatenar por columnas, `inner` conserva la intersección de índices y `outer` su unión.

Pandas: selección de datos

```
# %% Selección mediante posiciones: iloc
print(df.iloc[1])
print(df.iloc[0, 1])
print(df.iloc[:, 1])
print(df.iloc[:2, 1])

# %% Selección mediante etiquetas: loc
# En un índice entero, el extremo final de loc está incluido.
print(df.loc[:2, "ciudad"])
print(df.loc[:2, : "ciudad"])
```

`iloc` selecciona por posición y excluye el extremo final de un corte. `loc` selecciona por etiquetas e incluye el extremo final.

Ejercicio 8

Cargue `Churn_Modelling_sintetico_NANs.xlsx`.

1. Elimine las filas con valores perdidos.
2. Seleccione las variables de entrada que vea necesarias y prediga con ellas `EstimatedSalary`.
3. Compare LIN, SVR y Random Forest mediante MSE y validación cruzada de 10 folds.
4. Opcional: prediga `Exited` con tres clasificadores.
5. Opcional: sustituya la eliminación por métodos de imputación y repita la evaluación.
6. Discuta qué variables influyen en las predicciones.